

SECTION A

1. Difference between CSV and JSON Formats

CSV (Comma-Separated Values) and JSON (JavaScript Object Notation) are two commonly used formats for storing and exchanging data. CSV is a simple text format used mainly for storing tabular data in rows and columns, where each value is separated by a comma. It is widely used for spreadsheets, datasets, and database exports because it is lightweight and easy to read by humans and machines. In contrast, JSON stores data in a structured format using key–value pairs and supports hierarchical or nested data structures. This makes JSON more flexible for representing complex data relationships. While CSV is best suited for simple tabular datasets, JSON is commonly used in web applications, APIs, and systems that require structured data exchange.

2. Importance of Version Control (Git) in Data Science Projects

Version control is an essential tool in data science projects because it helps track changes in code, scripts, and project files over time. Git is one of the most widely used version control systems, allowing researchers and developers to manage different versions of their work efficiently. It enables multiple team members to collaborate on the same project without overwriting each other's changes. Git also helps maintain a clear history of modifications, making it possible to revert to earlier versions if errors occur. In data science workflows, Git is particularly useful for managing code related to data preprocessing, analysis, and model development. By using repositories such as GitHub or GitLab, researchers can also maintain backups of their projects and share their work easily with collaborators.

3. Resolving Merge Conflict Issues in GitHub

Merge conflicts occur in GitHub when two or more contributors modify the same part of a file and attempt to merge their changes into a single branch. When this happens, Git cannot automatically determine which version of the code should be kept. To resolve a merge conflict, the developer first pulls the latest changes from the repository and identifies the conflicting file. Git highlights the conflicting sections using special markers that show the different versions of the code. The developer must then manually review the conflicting parts and decide which changes to keep, modify, or combine. After editing the file and removing the conflict markers, the corrected file is saved, added to the staging area, and committed. Finally, the resolved version is pushed back to the repository, completing the merge process.

4. Imputation and Two Techniques

Imputation is a data preprocessing technique used to handle missing values in a dataset by replacing them with estimated values. Missing data can affect the performance of statistical analysis and machine learning models, so imputation helps maintain dataset completeness and usability. One common technique is mean imputation, where missing values are replaced with the average value of the corresponding variable. For example, if the ages in a dataset are 20, 25, and 30 with one missing value, the missing entry may be replaced with the mean value of 25. Another technique is median imputation, where missing values are replaced with the median (middle value) of the dataset. For instance, if income values include an extreme outlier, using the median instead of the mean helps produce a more reliable replacement value. These techniques help maintain data consistency and allow further analysis without removing valuable observations.

5. Feature Engineering and Its Importance

Feature engineering is the process of creating, selecting, or transforming variables from raw data to improve the performance of machine learning models. It involves extracting meaningful information from existing data so that algorithms can better identify patterns and relationships. Examples of feature engineering include creating new variables from existing data, normalizing numerical values, converting categorical variables into numerical formats, or deriving indices from satellite data in remote sensing studies. Feature engineering is important because the quality of features directly affects the accuracy and effectiveness of machine learning models. Well-designed features help models learn relevant patterns in the data, reduce noise, and improve predictive performance. In many data science applications, effective feature engineering can significantly enhance the results of analytical models.

SECTION B

Data Preprocessing and Feature Engineering in a Hydrology Dataset

1. Dataset Selection

For this assignment, a hydrology-related dataset titled “Daily Rainfall and River Discharge Dataset” was downloaded from Kaggle. The dataset contains information related to rainfall measurements, river discharge levels, temperature, soil moisture, and station location details. It includes both numerical variables (such as rainfall, discharge, temperature, and soil moisture) and categorical variables (such as station name and season).

The dataset contains more than 100 observations, which makes it suitable for analysis and modeling. It also contains missing values in some variables and extreme values in rainfall and discharge measurements, which can be considered potential outliers. These characteristics make the dataset appropriate for demonstrating data preprocessing techniques such as imputation, outlier detection, transformation, and feature engineering.

2. Loading the Dataset Using Pandas

The dataset was loaded into Python using the **pandas library**, which is widely used for data manipulation and analysis in data science.

```
import pandas as pd
# Load dataset
data = pd.read_csv("rainfall_discharge_dataset.csv")
# Display first few rows
print(data.head())
# Display dataset information
print(data.info())
# Statistical summary
print(data.describe())
# Check missing values
print(data.isnull().sum())
```

The `head()` function displays the first few rows of the dataset, which helps in understanding the structure of the data. The `info()` function provides information about the dataset including the number of entries, column names, and data types such as integers, floats, and objects (categorical variables). The `describe()` function summarizes the statistical properties of numerical variables such as mean, minimum, maximum, and standard deviation. The missing values were identified using the `isnull().sum()` function, which counts the number of missing entries in each column.

3. Imputation of Missing Values

To handle missing data, two imputation techniques were applied: mean imputation for numerical variables and mode imputation for categorical variables.

For numerical variables such as rainfall and soil moisture, mean imputation was used because it replaces missing values with the average value of the column, preserving the overall distribution of the dataset.

```
data['Rainfall'] = data['Rainfall'].fillna(data['Rainfall'].mean())
```

For categorical variables such as station name or season, mode imputation was used. Mode represents the most frequently occurring category and is suitable for replacing missing categorical values.

```
data['Season'] = data['Season'].fillna(data['Season'].mode()[0])
```

These techniques were selected because they are simple, effective, and commonly used in environmental datasets where missing observations occur due to sensor or recording issues.

4. Handling Outliers Using the IQR Method

Outliers were detected using the **Interquartile Range (IQR) method**, which identifies extreme values that lie far outside the typical range of the data.

```
Q1 = data['Rainfall'].quantile(0.25)
```

```
Q3 = data['Rainfall'].quantile(0.75)
```

```
IQR = Q3 - Q1
```

```
lower_bound = Q1 - 1.5 * IQR
```

```
upper_bound = Q3 + 1.5 * IQR
```

```
outliers = data[(data['Rainfall'] < lower_bound) | (data['Rainfall'] > upper_bound)]
```

```
print(outliers)
```

The IQR method identifies values that fall outside $1.5 \times \text{IQR}$ from the quartiles. In hydrological datasets, some extreme rainfall or discharge values may appear as outliers. However, these values may not necessarily be errors because heavy rainfall events or floods naturally produce extreme observations. Therefore, instead of removing them completely, they should be carefully examined and retained if they represent real environmental events.

5. Data Transformations

Two transformations were applied to improve the dataset for analysis and modeling.

Scaling

Numerical variables such as rainfall, discharge, and soil moisture were scaled using standardization, which transforms data to have a mean of zero and a standard deviation of one.

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
```

```
data[['Rainfall','Discharge']] = scaler.fit_transform(data[['Rainfall','Discharge']])
```

Scaling is necessary because machine learning algorithms perform better when variables are on similar scales.

Encoding Categorical Variables

Categorical variables such as Season were converted into numerical form using one-hot encoding.

```
data = pd.get_dummies(data, columns=['Season'])
```

Encoding is required because machine learning models cannot directly interpret categorical text values.

6. Feature Engineering

Two new features were created from the existing dataset to enhance analysis.

Rainfall Intensity

Rainfall intensity was derived from rainfall measurements and time duration. This feature helps in understanding how intense rainfall events affect river discharge.

```
data['Rainfall_Intensity'] = data['Rainfall'] / 24
```

Discharge to Rainfall Ratio

A ratio between discharge and rainfall was calculated to understand the relationship between precipitation and river response.

```
data['Discharge_Rainfall_Ratio'] = data['Discharge'] / (data['Rainfall'] + 1)
```

These engineered features provide additional insights into hydrological processes and can improve the predictive capability of models used for flood prediction or water resource analysis.